

Ofren Collect — Architecture Document

Project: Ofren Collect — Recurring billing + automatic reconciliation on Monnify **Author:** Ofren Technologies **Context:** APIConf Lagos 2026 × Monnify Developer Challenge **Stack:** React (frontend) · .NET 10 + SignalR (backend) · PostgreSQL (free, host-agnostic) · JWT auth + multi-tenancy · Monnify sandbox **Grounded in:** REQUIREMENTS.md, USER_STORIES.md

Version note: this document targets **.NET 10 (LTS, current)**. If your machines already have **.NET 8 (LTS)**, everything here applies unchanged — the APIs used are stable across both. Pick one and pin it in the README.

1. Overview

Ofren Collect is a two-tier web application: a **React single-page app** for the business owner's dashboard, and a **.NET Web API** backend that owns all business logic, persistence, and every interaction with Monnify. The backend is the only party that holds Monnify credentials and the only party that talks to Monnify — the React app never sees a secret.

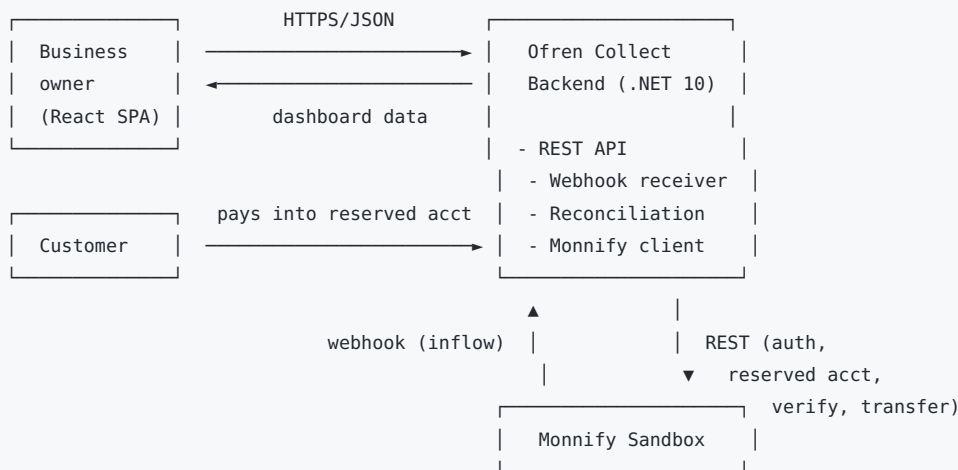
It is a **secure, multi-tenant** application: each business (tenant) signs in with JWT-based authentication and sees only its own data, enforced at the database layer. Every API call is **audit-logged** for full request/response visibility, the API is **rate-limited**, and all outbound calls to Monnify are wrapped in **retry and circuit-breaker** policies so a dependency failure degrades gracefully rather than cascading.

The system is event-driven at its core: payments arrive asynchronously as Monnify webhooks, are verified, reconciled against invoices, and the resulting state is **pushed live to the frontend over SignalR** — scoped per tenant — so the dashboard reflects a payment the instant it's processed, with no refresh. An **optional AI assistant** (\$9A) lets owners ask plain-language questions about their own collections; it is a clearly-bounded, cuttable stretch feature, not part of the core path.

1.1 Architectural drivers (from the requirements)

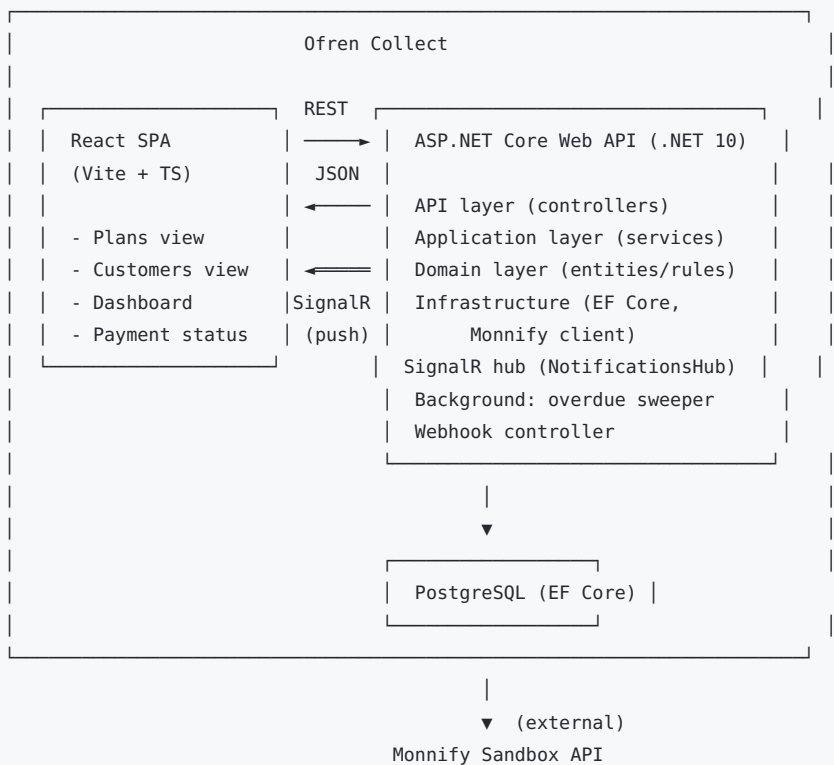
Driver	Source	Architectural consequence
Zero-touch reconciliation	FR-4.x	Reserved-account-per-subscription is the core identity mechanism
Never trust an unverified payment	FR-3.3, FR-3.4, NFR-1.4	Webhook auth + server-side verification before any state change
No double counting	FR-3.6, NFR-2.1	Idempotency keyed on Monnify transaction reference
No exposed secrets	NFR-1.1–1.3	Credentials only in backend config; frontend is credential-free
Judge can run it in minutes	NFR-3.1, NFR-4.1	PostgreSQL (free) via EF Core migrations + seed-on-start; single documented run path (local Postgres container or any managed Postgres)
Prompt webhook ack	FR-3.2, NFR-5.1	Receive → enqueue → 200, with processing decoupled from ack
Dashboard reflects payments in real time	FR-5.1, NFR-5.1	SignalR push: reconciliation broadcasts status changes to the SPA the instant they happen
Each business's data is isolated	multi-tenancy	Every tenant-owned entity carries a TenantId; a global query filter enforces isolation on every read; SignalR broadcasts are scoped to a per-tenant group
Only authenticated users act	auth	JWT bearer auth on the API and the SignalR hub; the tenant is derived from the token, never from client input
Full request/response visibility	audit	Middleware logs every API call — who, what, when, status, correlation id — to a queryable audit store
Survive transient dependency failures	resilience	Polly retry + circuit breaker around Monnify; rate limiting protects the API and webhook endpoint
Optional AI convenience without risk	FR-7.1, stretch	Natural-language query behind IAIAssistant + feature flag; read-only, tenant-scoped, grounded in real data; cuttable with one flag

2. System context (C4 level 1)



The customer never touches our UI — they simply pay into a bank account. That asynchronous inflow, relayed by Monnify as a webhook, is the system's primary input.

3. Container view (C4 level 2)



3.1 Why this split

The React SPA is a pure presentation client. All rules — reconciliation, status transitions, idempotency, verification — live in the backend where they can be tested and where secrets stay safe. This keeps the frontend replaceable and the trust boundary clean (NFR-1.x): the browser is untrusted, the API is the guardian.

4. Backend architecture (.NET 10)

4.1 Layered structure

A light Clean-Architecture-style layering — enough separation to be clear and testable, not so much it slows a 3-day build.

```
OfrenCollect.sln
|
├─ OfrenCollect.Api          (presentation: controllers, DI, middleware)
|   └─ Controllers/
|       ├── AuthController      (register, login → JWT)
|       ├── PlansController
|       ├── CustomersController
|       ├── SubscriptionsController
|       ├── DashboardController
|       ├── AuditController      (query audit log – admin)
|       ├── AssistantController (AI natural-language query – STRETCH, flag-gated)
|       └─ MonnifyWebhookController
|   └─ Hubs/
|       └─ NotificationsHub      (SignalR – pushes reconciliation events, [Authorize])
|   └─ Middleware/
|       ├── TenantResolutionMiddleware (token → ITenantContext)
|       └─ AuditLoggingMiddleware      (request/response capture)
|   └─ Program.cs                (composition root, pipeline, rate limiter, hub mapping)
|   └─ appsettings.json          (non-secret config only)
|
├─ OfrenCollect.Application    (use cases / services, DTOs, interfaces)
|   ├── Plans/
|   ├── Subscriptions/
|   ├── Reconciliation/        (the core engine)
|   ├── Auth/                  (registration, login, password hashing, JWT issuing)
|   ├── Assistant/             (AI query orchestration – STRETCH, flag-gated)
|   └─ Abstractions/           (IMonnifyClient, IReconciler, repos,
|                               IReconciliationNotifier ← push abstraction,
|                               ITenantContext, IAuditLogger, ICurrentUser,
|                               IAIAssistant ← provider-agnostic AI boundary)
|
├─ OfrenCollect.Domain          (entities, enums, domain rules – no dependencies)
|   ├── Tenant, User, Plan, Customer, Subscription, Invoice, PaymentEvent, AuditEntry
|   └─ Enums (InvoiceStatus, SubscriptionStatus, BillingInterval, UserRole)
|
└─ OfrenCollect.Infrastructure (EF Core, Monnify HTTP client, background jobs)
    ├── Persistence/           (Npgsql DbContext with global tenant query filter,
    |                           migrations, configs, seed, InboxMessage, AuditEntry)
    ├── Monnify/               (MonnifyClient + Polly retry + circuit breaker, options)
    ├── Auth/                  (JWT token service, password hasher)
    ├── Audit/                 (IAuditLogger → PostgreSQL sink)
    ├── Ai/                    (IAiAssistant impls: provider client + NullAiAssistant
    |                           – STRETCH, selected by config/feature flag)
    └─ Jobs/                   (OverdueSweeper, InboxDrainer : BackgroundService)
```

Dependency direction points inward: Api → Application → Domain; Infrastructure implements Application's interfaces. Domain depends on nothing. For a hackathon this can collapse into fewer projects if preferred — the *boundaries* matter more than the project count.

4.2 Request pipeline (middleware order)

Request

- HTTPS redirection
- CORS (allow the React dev origin; AllowCredentials for SignalR)
- Rate limiting (reject over-limit before any work; 429)
- Exception-handling middleware (uniform error JSON; never leak stack traces)
- Routing
- Authentication (validate JWT bearer; establish ClaimsPrincipal)
- Tenant-resolution middleware (read TenantId from token claims → ITenantContext)
- Authorization (is this user allowed here?)
- Audit middleware (capture request + response, correlation id, user, tenant)
- Endpoints:
 - Controllers (REST)
 - MapHub<NotificationsHub>("/hubs/notifications") (SignalR, [Authorize])

Response / persistent WebSocket connection

Order rationale: rate limiting sits first so flood traffic is shed before it costs anything. Authentication precedes tenant resolution because the tenant is derived from the validated token, never from a header or body the client controls (this is the core of tenant isolation — see §11.3). Audit wraps the endpoint so it records the authenticated user, resolved tenant, and the actual response status.

CORS note: SignalR's WebSocket handshake requires `AllowCredentials()` and an explicit origin (not `AllowAnyOrigin()`). Pin the React dev origin (e.g. `http://localhost:5173`) in the CORS policy, or the hub connection silently fails. JWTs for the hub are passed via the `access_token` query string on the WebSocket connection (SignalR's standard mechanism), validated by the same bearer scheme.

4.3 Key backend components

Controllers are thin — they validate input, call an application service, and map the result to an HTTP response. No business logic in controllers.

Application services hold use cases: `CreatePlan`, `EnrolCustomer`, `HandleTransactionNotification`, `GetDashboard`. Each maps to user stories (e.g. `EnrolCustomer` realises US-2.2/2.3).

The Reconciliation engine (`IReconciler`) is the heart (FR-4.x). It is deliberately isolated and pure where possible, so it can be unit-tested without HTTP or a database.

The Monnify client (`IMonnifyClient`) wraps all outbound calls: auth (with token caching), reserved-account creation, transaction verification, and transfers. It's an interface so the engine and services depend on an abstraction, and so field/endpoint corrections happen in one file (NFR-4.2). Its `HttpClient` is wrapped with **Polly** retry-with-backoff and timeout policies (free, via `IHttpClientFactory`) so transient Monnify failures don't break an enrolment or verification.

The overdue sweeper (`BackgroundService`) periodically flags overdue subscriptions (FR-4.8) — decoupled from the request path. When it flags a subscription, it also pushes the change (see below), so an invoice going overdue appears on the dashboard without a refresh.

The SignalR hub (`NotificationsHub`) is the real-time channel to the SPA. The reconciliation engine and the overdue sweeper never reference SignalR directly — they depend on `IReconciliationNotifier` (an Application abstraction), whose Infrastructure implementation wraps `IHubContext<NotificationsHub>`. This keeps the domain/application layers free of transport concerns while still enabling push. Events broadcast: `PaymentReconciled` (subscription id + new invoice status), `SubscriptionOverdue`, and `UnmatchedPayment`.

5. Frontend architecture (React)

5.1 Stack

- **Vite + React + TypeScript** — fast dev server, quick build, type safety for the API contract.
- **React Router** — a few routes: Dashboard, Plans, Customers, Subscription detail.
- **TanStack Query (React Query)** — server-state fetching and caching for the initial load and REST reads. Real-time updates arrive via SignalR (below) and are merged into the Query cache, so the two work together rather than competing.

- **@microsoft/signalr** — the official SignalR JavaScript client. Maintains a persistent connection to the backend hub and receives reconciliation events the moment they happen.
- **Fetch/axios** in a thin API client module — one place that knows the backend base URL.
- Minimal styling — a component library (e.g. a lightweight one) or hand-rolled CSS. Don't spend the budget here.

5.2 Structure

```
src/
├─ api/
│   └─ client.ts          (base URL, typed fetch wrappers)
├─ types/
│   └─ models.ts          (Plan, Subscription, Invoice – mirror backend DTOs)
├─ hooks/
│   └─ usePlans.ts
│   └─ useSubscriptions.ts
│   └─ useDashboard.ts
│   └─ useReconciliationHub.ts (SignalR connection; updates cache on push)
├─ realtime/
│   └─ hubConnection.ts    (builds/starts the SignalR connection, reconnect)
├─ pages/
│   └─ DashboardPage.tsx
│   └─ PlansPage.tsx
│   └─ CustomersPage.tsx
├─ components/
│   └─ StatusBadge.tsx     (colour-coded: paid/underpaid/overdue) – FR-5.2
│   └─ SubscriptionTable.tsx
│   └─ SummaryStrip.tsx    (collected / overdue / unmatched) – FR-5.3
└─ App.tsx
```

5.3 The live dashboard via SignalR

Reconciliation is asynchronous — a payment lands out-of-band, gets verified, and updates an invoice with no user action. To surface that instantly, the dashboard holds a **persistent SignalR connection** to the backend hub rather than polling.

Flow: on mount, `useReconciliationHub` starts a connection to `/hubs/notifications` and subscribes to the `PaymentReconciled`, `SubscriptionOverdue`, and `UnmatchedPayment` events. When the backend broadcasts one, the handler updates the relevant row in the React Query cache (or invalidates the dashboard query), and the affected subscription's status badge flips in place — the demo "money shot" of an invoice going to **Paid** happens the instant the webhook is processed, with no refresh and no polling delay.

```
Customer pays → Monnify webhook → backend verifies + reconciles
→ IReconciliationNotifier.PaymentReconciled(subscriptionId, status)
→ NotificationsHub broadcasts to connected clients
→ SPA receives event → updates cache → badge flips to Paid (live)
```

Resilience: the SignalR client is configured with automatic reconnection (`withAutomaticReconnect()`). On reconnect — or if a message is ever missed — the client refetches the dashboard once to resync, so the UI is never left stale. This makes the real-time layer an enhancement over a still-correct REST baseline, not a single point of failure: if the socket drops, a manual refresh (or the reconnect resync) still shows correct state.

Why SignalR over raw WebSockets: on a .NET backend, SignalR gives you connection management, automatic reconnection, and a transport fallback (WebSockets → Server-Sent Events → long polling) for free, with a clean typed hub API. Hand-rolling raw WebSockets would mean rebuilding all of that. SignalR is the idiomatic choice for the stack and the faster one to ship.

6. Data model

Tenant-owned entities plus the identity and audit tables. **Every business-data entity carries a `TenantId`** — this is the

backbone of isolation (§11.3).

```
Tenant
  Id, BusinessName, CreatedAt
  (the "business owner" account boundary – one tenant per business)

User
  Id, TenantId —————> Tenant
  Email (unique), PasswordHash, Role(enum: Owner|Staff)
  (a login; belongs to exactly one tenant)

Plan
  Id, TenantId —————> Tenant
  Name, Amount, Interval(enum), IsActive

Customer
  Id, TenantId —————> Tenant
  Name, Email

Subscription
  Id, TenantId —————> Tenant
  CustomerId —————> Customer
  PlanId —————> Plan
  ReservedAccountReference (our unique ref)
  ReservedAccountNumber, ReservedBankName (from Monnify)
  NextDueDate, Status(enum)

Invoice
  Id, TenantId —————> Tenant
  SubscriptionId —————> Subscription
  AmountDue, AmountPaid
  PeriodStart, DueDate
  Status(enum: Pending|Paid|Underpaid|Overpaid|Overdue)

PaymentEvent
  Id, TenantId (nullable ← unmatched inflows have no tenant yet)
  MonnifyTransactionReference (UNIQUE ← idempotency key)
  ReservedAccountNumber, Amount, PaidAt
  MatchedInvoiceId (nullable ← unmatched inflows)

AuditEntry
  Id, TenantId (nullable ← pre-auth calls), UserId (nullable)
  CorrelationId, Method, Path, QueryString
  RequestBody (sanitised), ResponseStatusCode, ResponseBodySummary
  DurationMs, IPAddress, TimestampUtc

(optional) SettlementTransfer
  Id, TenantId —————> Tenant
  Amount, MonnifyReference, Status, CreatedAt
```

6.1 Relationships

```

Tenant 1 — * User
Tenant 1 — * Plan / Customer / Subscription / Invoice    (isolation root)

Customer 1 — * Subscription * — 1 Plan
              |
              1
              |
              *
              Invoice 1 — * PaymentEvent (via matched account/invoice)

```

6.2 The decisions that make reconciliation work

1. **PaymentEvent.MonnifyTransactionReference is unique** — the database itself enforces idempotency (NFR-2.1). A re-delivered webhook hits a unique-constraint and is safely ignored.
2. **ReservedAccountNumber is the match key**, never the amount. Two customers on the same ₦5,000 plan are told apart by the account the money landed in (this is why TC-4.10 passes).
3. **Reserved accounts are globally unique across tenants**, so an inflow resolves to exactly one subscription and therefore one tenant — the webhook itself is tenant-agnostic (Monnify doesn't know our tenants), but the reserved account it paid into deterministically identifies the owning tenant. This is why the webhook path is the one place tenant context comes from data, not from a token (§11.3).

7. Key flows

7.1 Enrol a customer (US-2.2, US-2.3)

```

React: POST /api/subscriptions { customerId, planId }
→ SubscriptionsController
→ EnrolCustomer service
  → create Subscription (generate ReservedAccountReference)
  → IMonnifyClient.CreateReservedAccount(...) [outbound to Monnify]
  → persist returned account number + bank
  → create first Invoice (amount = plan amount, dueDate from interval)
→ return subscription DTO (incl. reserved account number)
React: display the account number for the owner to share

```

If Monnify account creation fails, the whole operation fails cleanly and no half-built subscription is persisted (TC-2.8) — wrap in a transaction.

7.2 Payment received & reconciled (US-3.2, US-3.3, US-4.x) — the core

```

Monnify → POST /api/webhooks/monnify (raw body + signature header)
MonnifyWebhookController:
  1. read raw body
  2. verify signature/hash — invalid → 401, stop (FR-3.3)
  3. enqueue for processing
  4. return 200 immediately (FR-3.2, NFR-5.1)

Processing (HandleTransactionNotification):
  5. extract transaction reference
  6. if reference already in PaymentEvent → stop (idempotent) (FR-3.6)
  7. IMonnifyClient.VerifyTransaction(ref) — verify server-side (FR-3.4)
  8. resolve Subscription by ReservedAccountNumber
     | (this also determines the owning TenantId — see §11.3)
     └ none? record PaymentEvent as Unmatched (TenantId null), surface it (FR-4.7)
  9. apply amount to current open Invoice
     └ == due → Paid; advance subscription; new invoice (FR-4.3, FR-4.6)
     └ < due → Underpaid; record balance (FR-4.4)
     └ > due → Overpaid; record credit (FR-4.5)
  10. persist PaymentEvent (reference unique → idempotency; TenantId set)
  11. IReconciliationNotifier.PaymentReconciled(tenantId, subId, status) (FR-5.1)
     → NotificationsHub broadcasts to GROUP "tenant-{tenantId}" only
     → that tenant's SPA badge flips live; no other tenant sees it

```

Step 11 fires only after the state is durably persisted (step 10), so the push always reflects committed truth — a client can never be told "Paid" for a transaction that failed to save. The broadcast targets a **per-tenant SignalR group**, so a payment for one business never appears on another's dashboard (§11.3).

7.3 Overdue sweep (US-4.7)

```

OverdueSweeper (BackgroundService), runs on an interval:
  find invoices past DueDate not fully paid
  → mark subscription Overdue (FR-4.8)
  → IReconciliationNotifier.SubscriptionOverdue(subId) (FR-5.1)
  → NotificationsHub broadcasts → dashboard flags it live

```

7.4 Dashboard read + live updates (US-5.x)

```

Initial load:
React (React Query): GET /api/dashboard
  → DashboardController
  → project subscriptions + statuses + summary (collected/overdue/unmatched)
  → return DTO
React: render table + colour-coded badges + summary strip

Live thereafter:
React starts SignalR connection to /hubs/notifications (JWT in access_token)
  → hub reads TenantId from the token, joins caller to group "tenant-{tenantId}"
  → on PaymentReconciled / SubscriptionOverdue / UnmatchedPayment (tenant-scoped):
     update React Query cache → affected badge/summary updates in place
  → on disconnect: withAutomaticReconnect; on reconnect: refetch to resync

```

The REST read establishes correct initial state; SignalR keeps it live. The two are complementary — the dashboard is fully correct even if the socket never connects, and instant when it does.

8. API surface (REST)

Method	Route	Purpose	Auth	Story
POST	/api/auth/register	Create a tenant + owner user	Anonymous	auth
POST	/api/auth/login	Authenticate → JWT (with tenant claim)	Anonymous	auth
POST	/api/plans	Create a plan	Bearer	US-1.1
GET	/api/plans	List plans	Bearer	US-1.2
POST	/api/customers	Register a customer	Bearer	US-2.1
POST	/api/subscriptions	Enrol customer (provisions reserved account)	Bearer	US-2.2
GET	/api/subscriptions	List subscriptions	Bearer	US-5.1
GET	/api/subscriptions/{id}	Subscription detail + invoices	Bearer	US-5.1
POST	/api/subscriptions/{id}/cancel	Cancel a subscription	Bearer	US-2.4
GET	/api/dashboard	Dashboard projection + summary	Bearer	US-5.1, US-5.3
GET	/api/audit	Query audit log (own tenant)	Bearer (Owner)	audit
POST	/api/assistant/ask	AI natural-language collections query (STRETCH)	Bearer (flag-gated)	FR-7.1
POST	/api/webhooks/monnify	Receive Monnify notifications	Signature (no JWT)	US-3.2
POST	/api/subscriptions/{id}/settle	(stretch) settle to merchant	Bearer	US-6.1
WS	/hubs/notifications	SignalR hub — real-time reconciliation events	Bearer	US-5.1

All REST request/response bodies are JSON DTOs; entities are never exposed directly. All **Bearer** routes require a valid JWT and operate strictly within the token's tenant. The webhook is the deliberate exception — it's called by Monnify, not a logged-in user, so it's authenticated by **signature verification** instead of a JWT, and it resolves its tenant from the reserved account (§11.3). The SignalR hub is a persistent WebSocket connection (with SSE/long-poll fallback), not a REST route, so it's listed separately.

8.1 SignalR hub contract

The hub is server-to-client push only (the SPA subscribes; it doesn't invoke server methods). Events:

Event	Payload	Fired when
PaymentReconciled	subscriptionId, invoiceId, status, amountPaid	A verified payment updates an invoice
SubscriptionOverdue	subscriptionId	The sweeper flags a subscription overdue
UnmatchedPayment	reference, amount, accountNumber	An inflow matches no subscription

9. Monnify integration boundary

All Monnify interaction is behind **IMonnifyClient** in Infrastructure. Four operations:

Operation	Used by	Requirement
AuthenticateAsync (Basic → bearer, cached)	all calls	NFR-2.4
CreateReservedAccountAsync	EnrolCustomer	FR-2.3
VerifyTransactionAsync	reconciliation	FR-3.4
InitiateTransferAsync (stretch)	settlement	FR-6.1

The concrete endpoints, request shapes, and the webhook signature scheme must be confirmed against the live Monnify sandbox before coding — treat the sandbox docs as authoritative over any snippet. The interface boundary means such corrections touch one class. See ARCHITECTURE.md (the build guide) for the C# implementation sketches of auth, reserved-account creation, the webhook receiver, and verification.

Registration: **IMonnifyClient** via **IHttpClientFactory**, wrapped with a **Polly resilience pipeline — retry-with-backoff, timeout, and a circuit breaker** (opens after consecutive failures so we stop hammering a down Monnify and fail fast, then half-open to test recovery). **MonnifyOptions** bound from configuration; secrets from user-secrets/environment only.

9A. AI assistant — natural-language collections query (STRETCH, cuttable)

Status: optional stretch. This entire capability sits behind a feature flag and a single interface. If it isn't solid by submission, it is switched off and the product loses nothing — every answer it gives is also reachable through the normal dashboard. It is built *last*, after the P1 reconciliation slice and the security layer are working. Included here because the brief encourages AI, but deliberately scoped so it can never become the "AI slop" the brief warns against.

9A.1 What it does (and what it deliberately doesn't)

The assistant lets a business owner ask plain-language questions about **their own collections data** and get a grounded answer:

- "How much did I collect this week?"
- "Which customers are overdue?"
- "Who underpaid last month, and by how much?"

It is a **read-only reporting convenience**, not an agent. It cannot create plans, move money, cancel subscriptions, or change any state — it only turns a question into a safe, tenant-scoped query and phrases the result. That boundary is the single most important design decision here: it keeps the feature genuinely useful while making a whole class of failure modes (destructive actions, hallucinated changes) structurally impossible.

9A.2 Why it isn't "slop"

The brief's bar is *useful, practical, well-designed* — not "uses AI." This clears that bar because:

- **It answers a real question a non-technical owner actually asks**, without making them learn dashboard filters.
- **Every answer is grounded in real data**, not model memory — the model never invents figures (see the query-translation design below).
- **It degrades to nothing**, not to a broken experience — if disabled, the dashboard already shows all the same facts.

If it can't be made to do this reliably, the correct move is to cut it (per the requirements' FR-7.2 governance rule), and the architecture makes cutting a one-line flag change.

9A.3 Design — grounded, not free-form

Two safe patterns, in preference order. **Neither ever lets the model touch the database directly.**

Pattern A — intent + parameters (preferred, safest). The model's only job is to classify the question into one of a small set of **predefined, parameterised queries** and extract parameters (date range, status). The actual data fetch is ordinary, tenant-filtered application code.

```
"how much did I collect this week?"
→ LLM classifies → intent = CollectedTotal, range = thisWeek (structured JSON)
→ app runs the existing tenant-scoped query for CollectedTotal(thisWeek)
→ app formats the number into a sentence (optionally via the LLM for phrasing)
```

The model sees a question and returns a small JSON object; it never sees raw SQL and never returns figures of its own. An unrecognised question returns "I can't answer that yet" — not a guess.

Pattern B — natural-language-to-summary over a pre-fetched result set. For slightly richer phrasing, the app fetches the tenant-scoped data first, then asks the model to summarise *that specific, supplied data*. The model is given the numbers; it only narrates them. It cannot fetch, so it cannot leak across tenants or invent totals.

Both patterns mean **tenant isolation is never delegated to the model** — the same `ITenantContext` + global query filter (§11.3) that guards every other read guards these too. The AI sits on top of the safe data layer, never beside it.

9A.4 The provider boundary (provider-agnostic)

All model interaction is behind one interface in the Application layer:

```
public interface IAiAssistant
{
    Task<AssistantAnswer> AskAsync(string question, TenantId tenant, CancellationToken ct);
}
```

- The **Application/Domain layers depend only on** `IAiAssistant` — they know nothing about any vendor.
- **Infrastructure supplies the implementation**, chosen by configuration: a hosted API (e.g. Claude or OpenAI) or a **free local model** (e.g. Ollama) for a no-cost, no-external-call demo. Swapping providers is a DI registration change, not a code change.
- A `NullAiAssistant` implementation returns "assistant unavailable" and is what's registered when the feature flag is off — so the rest of the app compiles and runs identically with AI removed.
- The provider call is wrapped in the **same Polly retry + circuit breaker** treatment as Monnify, and **rate-limited per tenant** so the assistant can't run up cost or load.

9A.5 Safety, cost, and privacy notes

- **No secrets or PANs in prompts.** Only the minimal, already-tenant-scoped data needed to answer goes to the model; nothing sensitive beyond the figures themselves, and never Monnify credentials.
- **Provider key** lives in configuration/secrets like every other credential — never in the repo, never in the frontend.
- **Feature flag + kill switch:** `Ai:Enabled=false` disables the endpoint and registers `NullAiAssistant`. This is the cut mechanism.
- **Audit + rate limit apply:** assistant calls go through the same audit middleware and rate limiter as any other endpoint.

9A.6 Endpoint

Method	Route	Purpose	Auth
POST	/api/assistant/ask	Ask a natural-language question about own collections	Bearer (flag-gated)

Returns a short answer plus, where relevant, the structured figures behind it — so the UI can show both the sentence and a link to the matching dashboard view (reinforcing that the AI adds convenience, not a separate source of truth).

10. Cross-cutting concerns

Configuration & secrets (NFR-1.1-1.3): non-secret settings in `appsettings.json`; `Monnify:ApiKey/SecretKey/ContractCode` in user-secrets locally or environment variables. `.env.example` documents the shape; real values never committed. `.gitignore` blocks `.env` and secrets.

Authentication (JWT): users register/login via `AuthController`; passwords hashed with a strong KDF (ASP.NET Core `PasswordHasher` / PBKDF2 — free). Login issues a signed **JWT** carrying `sub` (user id), `tenant_id`, and `role` claims. The API and the SignalR hub both require a valid bearer token; the hub reads the token from the `access_token` query string on connect. The signing key lives in configuration/secrets, never the repo.

Multi-tenancy (isolation): covered in depth in §11.3. In short: every tenant-owned entity has a `TenantId`; an **EF Core global query filter** appends `WHERE TenantId = @current` to every read automatically, so a developer cannot accidentally leak across tenants; the current tenant comes from the JWT via `ITenantContext`, never from client input; and SignalR broadcasts go to per-tenant groups.

Audit logging (full request/response visibility): `AuditLoggingMiddleware` wraps every API call and writes an `AuditEntry` to PostgreSQL capturing a correlation id, the authenticated user and tenant, HTTP method and path, a **sanitised** request body, the response status code and a response summary, duration, and client IP. Sensitive fields (passwords, Monnify secrets, tokens, full PANs) are redacted before persistence — visibility must not become a secret-leak (NFR-1.6). The correlation id is also emitted on the response header and in structured logs, so a single request can be traced across audit store and logs. Owners can query their own tenant's audit trail via `GET /api/audit`; the audit query is itself tenant-filtered.

Rate limiting: the built-in .NET **rate limiter middleware** (free, `Microsoft.AspNetCore.RateLimiting`) protects the API. Two policies: a **per-tenant/per-user** limit on authenticated endpoints (fair use across tenants) and a tighter **per-IP fixed-window** limit on the anonymous auth endpoints and the public webhook, to blunt brute-force and flooding. Over-limit requests get `429 Too Many Requests` with a `Retry-After` header, shed before they reach controllers.

Resilience (retry / circuit breaker): all outbound Monnify calls run through a **Polly** pipeline — retry with exponential backoff for transient faults, a timeout, and a **circuit breaker** that trips after repeated failures (fail fast instead of piling

requests onto a struggling dependency, then half-open to probe recovery). This keeps a Monnify outage from cascading into hung enrolments or webhook processing.

Error handling (NFR-2.2): one exception-handling middleware returns uniform JSON errors and never leaks internals. Monnify failures surface as controlled errors, never crashes.

Idempotency (NFR-2.1): enforced at the database via the unique transaction reference, plus an application-level pre-check — belt and braces against races (TC-3.9).

Logging (NFR-1.6): structured logging (built-in `ILogger`) with correlation ids; never log full secrets or full sensitive payloads. Log transaction references and outcomes for demo-time visibility.

CORS: backend allows the React dev origin (e.g. `http://localhost:5173`) so the SPA can call the API in development. The policy must set `AllowCredentials()` with an explicit origin (SignalR's WebSocket handshake rejects `AllowAnyOrigin()`), and the SignalR endpoint is included in the same policy.

Real-time push (FR-5.1): reconciliation and overdue events reach the SPA over SignalR via the `IRconciliationNotifier` abstraction, scoped to per-tenant groups. Broadcasts happen only after state is persisted, and the client resyncs on reconnect — so push is an enhancement layered over a correct REST baseline, never the sole source of truth.

Validation: input validated at the controller/DTO boundary (data annotations or a validator); invalid plans/customers rejected with clear messages (TC-1.3–1.5, TC-2.2).

11. Scope, resilience, and deployment decisions

This build targets a **single backend instance**, uses **PostgreSQL** (free, host-agnostic) as the datastore, and relies only on **free and open-source components**. That decision resolves most "at scale" questions directly — because we are explicitly not running multiple instances, the machinery that exists only to coordinate across instances is not needed. The items below are folded *into* the build rather than deferred; the short list at the end is what genuinely remains out of scope, and why.

11.1 Included in this build

- **PostgreSQL as the datastore.** PostgreSQL is free, open-source, and production-grade well beyond this scale. It is **host-agnostic**: locally it runs as the official `postgres` **Docker image** (one command, zero licensing), and in the cloud it runs on any managed Postgres — Neon, Supabase, Railway, Render, Aiven — with only a connection-string change. EF Core talks to it via `Npgsql.EntityFrameworkCore.PostgreSQL` (free). Schema is created by **EF Core migrations** on startup and seeded so the app is never empty (NFR-3.2, NFR-4.1). Because we standardised on PostgreSQL from the start, the demo and the deployed datastore are the same engine — no provider swap, and no lock-in to any one host.
- **Durable webhook handling (transactional outbox/inbox), single-instance flavour.** Webhook processing does not need an external message broker at one instance. Instead we use a lightweight **inbox/outbox pattern inside PostgreSQL**: the raw verified notification is written to an `InboxMessage` table in the *same database transaction* that records the `PaymentEvent`, so a crash mid-processing cannot lose an acknowledged payment — on restart, unprocessed inbox rows are picked up and completed. This gives the durability a broker would provide, using only the database you already have. The unique transaction reference still enforces idempotency (NFR-2.1).
- **Retries and resilience on Monnify calls (Polly).** Polly is free and integrates with `IHttpClientFactory`. The Monnify `HttpClient` is wrapped with a **retry-with-exponential-backoff** policy for transient failures and a **timeout** policy, so a momentary Monnify hiccup doesn't fail an enrolment or a verification outright. This is standard `Microsoft.Extensions.Http.Polly` wiring — no paid dependency.
- **SignalR real-time push, single-instance.** With one backend instance, SignalR needs **no backplane** — a broadcast from the reconciler reaches every connected client directly, because they're all connected to the same instance. This is not a limitation here; it's the correct, simplest configuration for a single-instance deployment. (The only thing a Redis backplane would add is cross-instance fan-out, which single-instance doesn't need.)
- **Background processing (in-process hosted services).** The overdue sweeper and the inbox drainer run as `BackgroundService` hosted services inside the same process — free, built into .NET, and entirely sufficient for one instance. No external scheduler or worker tier.
- **Authentication (JWT) and RBAC.** Registration/login with hashed passwords (ASP.NET Core `PasswordHasher`, free) issuing signed JWTs. Roles (`Owner`, `Staff`) gate sensitive actions like viewing the audit log. All free, built into `Microsoft.AspNetCore.Authentication.JwtBearer`.

- **Multi-tenancy.** Full tenant isolation across data and real-time channels — see §11.3 for the mechanism.
- **Audit logging.** Every API call recorded to PostgreSQL with correlation id, user, tenant, request/response and timing (sanitised). Queryable per tenant. Built with middleware and EF Core — no external service.
- **Rate limiting.** Built-in .NET rate limiter middleware — per-tenant/user limits on authenticated routes, tighter per-IP limits on auth and webhook endpoints. Free, first-party.
- **Retry + circuit breaker (Polly).** Outbound Monnify calls wrapped in retry-with-backoff, timeout, and a circuit breaker via `Microsoft.Extensions.Http.Polly` (free) — transient faults are retried, a sustained outage trips the breaker to fail fast.
- **AI assistant (stretch, flag-gated).** A read-only natural-language query over the owner's own collections data, behind `IAiAssistant` and a feature flag (\$9A). Provider-agnostic — a free local model (Ollama) keeps it zero-cost; a hosted API is a config swap. Off by default is a one-line change and removes nothing the dashboard doesn't already show. Built last; cut first if the P1 slice needs the time.

11.2 Genuinely out of scope (single-instance / demo reasoning)

- **Horizontal scale-out (multiple instances).** Explicitly not a goal. The two things multi-instance would require — a **Redis backplane** for SignalR and a shared/distributed cache — are therefore not included. If this ever needed to scale out, those are the known, additive steps; nothing in the current design blocks them (the notifier is already an abstraction, the tenant context is already resolved per request, EF Core is already provider-consistent).
- **External identity provider / SSO / refresh-token rotation.** Auth here is self-issued JWTs with a reasonable lifetime — enough for a real, secure multi-tenant demo. Production hardening (refresh-token rotation, OIDC/social login, MFA) is additive and doesn't change the architecture.
- **Tenant onboarding beyond self-service register.** A tenant is created by the register endpoint; billing/plan-tier management for the *tenants themselves* (as opposed to their customers) is out of scope.

Documenting this shows architectural maturity: the build is internally consistent for its stated target (one instance, free tools, PostgreSQL, secure multi-tenant), and the few genuine exclusions are tied to a clear reason rather than left unexplained.

11.3 Multi-tenancy — how isolation is guaranteed

Tenant isolation is a correctness-and-security property, so it's enforced in **layers** rather than trusting any single check:

1. **Every tenant-owned entity carries a `TenantId`.** Plans, customers, subscriptions, invoices, settlements — all stamped with the owning tenant.
2. **The current tenant comes from the validated JWT, never from client input.** `TenantResolutionMiddleware` reads the `tenant_id` claim from the authenticated token and populates a scoped `ITenantContext`. A client cannot ask for another tenant's data by passing an id — there's no such parameter; the tenant is whoever the token says they are.
3. **An EF Core global query filter enforces isolation automatically.** The `DbContext` applies `HasQueryFilter(e => e.TenantId == _tenantContext.TenantId)` to every tenant-owned entity. Every LINQ query is silently scoped — a developer who writes `db.Subscriptions.ToList()` gets only the current tenant's rows. Isolation is the default, not something each query must remember to add. On save, `TenantId` is stamped automatically in `SaveChanges` so it can't be forgotten or forged.
4. **SignalR broadcasts are tenant-scoped.** On connect, the hub places each connection into a group named `tenant-{tenantId}` (from the token). Reconciliation events are broadcast to that group only, so a payment for one business never surfaces on another's live dashboard.
5. **The webhook is the one tenant-agnostic entry point — and that's safe.** Monnify doesn't know our tenants, so the webhook can't carry a tenant token. Instead, the reserved account the money landed in deterministically identifies the subscription and therefore the tenant (reserved accounts are globally unique). The webhook authenticates by **signature**, resolves the tenant from data, and stamps the resulting records accordingly. An inflow matching no account is stored tenant-less as `Unmatched` for review.

This layered approach means a single mistake doesn't breach isolation: even if a query forgot its filter (it can't — it's global), the tenant context still comes only from a signed token, and the write path stamps tenant independently.

12. Deployment / run topology (local)

```
Datastore: PostgreSQL – docker run postgres OR any managed Postgres (Neon/Supabase/Railway/etc.)
Terminal 1: dotnet run --project src/OfrenCollect.Api      → https://localhost:5001
Terminal 2: npm run dev  (in /web)                       → http://localhost:5173
Terminal 3: ngrok http 5001  → public URL for Monnify webhook
```

The PostgreSQL schema is created by EF Core migrations and seeded on first backend run (NFR-3.2, NFR-4.1) — run against a **local Postgres container** (`docker run postgres` , zero setup) or point the connection string at **any managed Postgres** (Neon, Supabase, Railway, etc.); the README documents both. The React app's API base URL and SignalR hub URL both point at the backend (`https://localhost:5001` , hub at `/hubs/notifications`). For submission simplicity you *may* serve the built React app as static files from the .NET app (single process, single URL) — note this option in the README; it makes the judge's run a one-command experience and means the SPA and hub share an origin, sidestepping CORS entirely.

Demo tip: because the dashboard now updates over SignalR, you can keep it open on screen while you trigger a sandbox payment from another window — the badge flips live in front of the judges. Keep the fallback clip (see the demo script) in case the hub connection is slow on conference Wi-Fi.

13. Traceability (architecture → requirements)

Component	Realises
Reserved-account-per-subscription	FR-2.3, FR-4.1
Webhook controller (verify + 200 fast)	FR-3.1, FR-3.2, FR-3.3
Reconciliation engine	FR-4.1-4.6
Unique transaction reference	FR-3.6, NFR-2.1
Overdue sweeper	FR-4.8
Dashboard projection (REST) + SignalR push	FR-5.1-5.3, NFR-5.1
NotificationsHub + IReconciliationNotifier	FR-5.1 (real-time)
IMonnifyClient boundary	NFR-4.2
Config/secrets handling	NFR-1.1-1.3
PostgreSQL (EF Core migrations) + seed	NFR-4.1, NFR-3.2
Inbox/outbox in PostgreSQL (durable webhooks)	FR-3.2, NFR-2.1, NFR-2.3
Polly retry + circuit breaker on Monnify client	NFR-2.2
JWT auth (register/login, hashed passwords)	auth
Multi-tenancy (TenantId + global query filter + tenant groups)	multi-tenancy
Audit logging middleware → AuditEntry store	audit
Rate limiter middleware (per-tenant + per-IP)	resilience
IAiAssistant + NullAiAssistant (flag-gated)	FR-7.1, FR-7.2 (stretch)
Exception middleware	NFR-2.2

14. Architecture decisions (quick log)

Decision	Why	Trade-off accepted
React SPA + separate .NET API	Clean trust boundary; frontend holds no secrets	Two dev servers locally (or serve static from .NET)
.NET 10 LTS	Current, supported to 2028	.NET 8 equally fine if already installed
PostgreSQL (host-agnostic)	Free, open-source, production-grade; runs anywhere (local container or any managed Postgres); no lock-in	Needs a Postgres instance running (vs a single file)
Inbox/outbox in the DB (not a broker)	Durable webhook handling without external infra; fits single-instance	Manual pattern; not cross-instance (not needed here)
Polly retry + circuit breaker for Monnify	Free resilience via IHttpClientFactory; fail fast on sustained outage	Slightly more HTTP wiring; breaker tuning
JWT auth (self-issued)	Free, first-party, sufficient for secure multi-tenant demo	No SSO/refresh rotation (documented)
Multi-tenancy via global query filter	Isolation is the default, not per-query discipline; hard to leak	Every entity needs TenantId; tenant context per request
Audit via middleware → PostgreSQL	Full request/response visibility, queryable, no external service	Storage grows; bodies must be sanitised
Built-in .NET rate limiter	Free, first-party; per-tenant + per-IP policies	Limits need tuning to avoid false 429s
AI assistant behind IAiAssistant + flag	Encouraged by brief; genuinely useful yet fully cuttable; provider-agnostic	Extra interface + impl; built last, cut first
AI read-only, intent-classification design	Grounded in real data; no destructive actions or hallucinated figures possible	Limited to predefined question types
SignalR for real-time push	Idiomatic on .NET; instant dashboard updates; free reconnection + transport fallback	Single-instance by design — no backplane needed
Notifier abstraction over the hub	Keeps domain/application free of transport concerns; testable	One extra interface + Infrastructure impl
Push layered over REST baseline	Dashboard stays correct if socket drops; resync on reconnect	Slightly more client logic than pure polling
In-process webhook handling	Simpler; idempotency covers safety	No durable queue; documented
Reserved account = identity	Enables zero-touch reconciliation	One account provisioned per subscription
Layered but few projects	Testable core without ceremony	Less separation than full enterprise Clean Arch